

SEMINAR PRESENTATION

CSA1002-SOFTWARE ENGINEERING



A SOFTWARE TESTING TECHNIQUE

Mutation Testing

Checking whether our test cases are strong enough to find errors in a program.

**Guided by
Dr. Sharmili K**

**Presented by
R. Pranathi-192525076**



What is Software Testing?



Checking whether software works correctly



Finding errors or bugs



Checking whether the software gives the expected output



Making the software more reliable

SIMPLE EXAMPLE

A calculator

Enter: 5 + 5

Expected answer: 10



If the calculator gives **15**, there is a problem.

"Software testing helps us find problems like this before the software is given to users."



Why Do We Need Mutation Testing?

Suppose we have a program and 10 test cases — **all 10 pass**. Can we say testing is perfect? **No!**

A program should accept: $\text{Age} \geq 18$

OUR TEST CASES CHECK



Age = 20



Age = 25



Age = 30

NEVER TESTED



Age = 18

The exact boundary value — so we may miss an important error.

Mutation testing helps us find whether our test cases are really good or not.



What is Mutation Testing?

Making small changes in a correct program and checking whether our test cases can find those changes.

THE CHANGED PROGRAM IS CALLED A

Mutant

Simple way to think about it:



We intentionally put a small mistake into the program and check whether our test cases can catch it. (Don't worry about the word “mutation”!)



Simple Idea

ORIGINAL PROGRAM

$10 + 5 = 15$

MUTANT (small change)

$10 - 5 = 5$



Run the test case

Expected: **15**

Mutant gives: **5**

The test case detects the mistake.

MUTANT = KILLED



Important Terms

Term	Simple Meaning
Original Program	Correct program
Mutant	Program with a small change
Mutation	The change we make
Test Case	Test used to check the program
Killed Mutant	Test found the mistake
Survived Mutant	Test did not find the mistake
Equivalent Mutant	Change does not affect the result



OUTCOMES

Killed Mutant

ORIGINAL

```
if age >= 18:  
    Eligible
```

MUTANT

```
if age > 18:  
    Eligible
```

Test: Age = 18

RESULT



Original: **18 >= 18** → **TRUE (Eligible)**



Mutant: **18 > 18** → **FALSE (Not eligible)**

“Killed” doesn't mean the program is destroyed — it means our test case detected the change.

MUTANT IS KILLED



OUTCOMES

Survived Mutant

ORIGINAL

```
if age >= 18:
```

MUTANT

```
if age > 18:
```

Our test cases only use:

Age = 20

Age = 25

Age = 30

For all these values, both conditions give the SAME result:

`age >= 18` $\equiv$ `age > 18`

The test cases don't notice the change — our test cases may be weak.

MUTANT SURVIVES



Mutation Operators

Rules used to make small changes in the program.



Change arithmetic
operator

`a + b`



`a - b`



Change comparison

`>`



`>=`



Change equality

`==`



`!=`



Change logical operator

`AND`



`OR`



Change a value

`age = 18`



`age = 19`



Example of Mutation Operators

ORIGINAL

```
if marks >= 40:  
    Pass
```



Mutant 1

```
if marks > 40:
```

>= → >

Mutant 2

```
if marks == 40:
```

>= → ==

Mutant 3

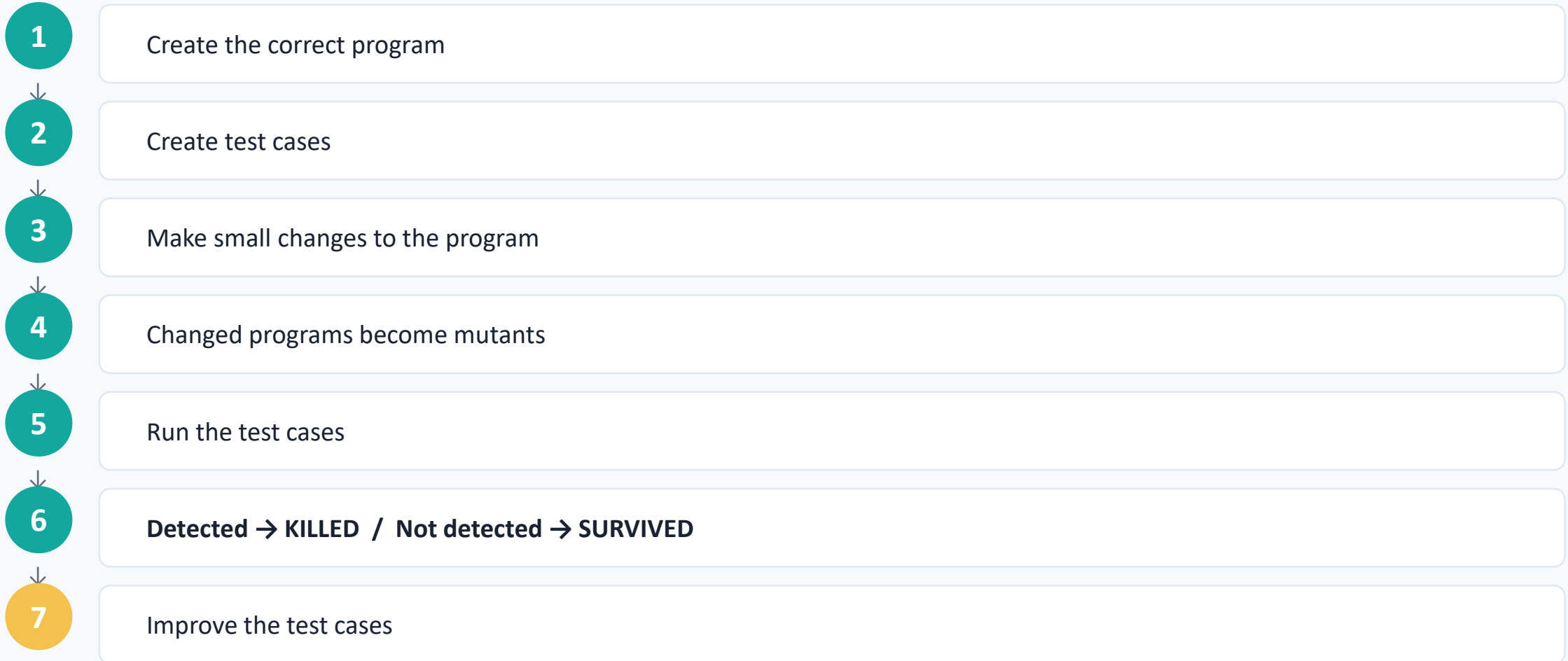
```
if marks < 40:
```

>= → <

Each changed version is a mutant.



Mutation Testing Process





Simple Flowchart





Mutation Score

Tells us how many mutants our tests were able to find.

$$\text{Mutation Score} = (\text{Killed Mutants} \div (\text{Total} - \text{Equivalent})) \times 100$$

Higher mutation score → stronger test suite

SIMPLE EXAMPLE

Total mutants	10
Killed	8
Equivalent	1

$$(10 - 1) = 9 \rightarrow 8 \div 9 \times 100$$

88.89%



REAL-TIME EXAMPLE 1

ATM Mutation

ORIGINAL

```
balance >= amount
```

MUTANT

```
balance > amount
```

Test: Balance = ₹10,000, Withdrawal = ₹10,000

RESULT



Original: **10000** >= 10000 → **TRUE (Allowed)**



Mutant: **10000** > 10000 → **FALSE (Rejected)**

We changed >= to > and the exact boundary amount exposed the mistake.

MUTANT IS KILLED



REAL-TIME EXAMPLE 1

ATM Withdrawal

The ATM should allow withdrawal when:

Balance $\geq$ Withdrawal Amount

For example: Balance = ₹10,000 Withdrawal = ₹10,000 — the customer should be allowed to withdraw.

ORIGINAL PROGRAM

```
if balance >= amount:  
    Allow withdrawal  
else:  
    Reject
```





What Did We Learn From the ATM Example?

The important point is:

Boundary Testing

We tested the exact boundary:

₹10,000 = ₹10,000

If we had only tested:

₹5,000

₹7,000

₹8,000

...the mutant might have SURVIVED, and we'd never have known our test suite was weak.

Mutation testing can show us: "You need better test cases."



REAL-TIME EXAMPLE 2

Online Shopping Discount

The website gives a 10% discount when the purchase is ₹5,000 or more.

ORIGINAL PROGRAM

```
if amount >= 5000:  
    Give 10% discount
```





REAL-TIME EXAMPLE 2

Shopping Mutation

ORIGINAL

```
if amount >= 5000:  
    Give 10% discount
```

MUTANT

```
if amount > 5000:  
    Give 10% discount
```

Test: Purchase = ₹5,000 — a developer accidentally changed the condition

RESULT



Original: **5000 >= 5000** → **TRUE** (Gets 10% discount)



Mutant: **5000 > 5000** → **FALSE** (No discount)

The test detects the error the developer accidentally introduced.

MUTANT IS KILLED



SPECIAL CASE

Equivalent Mutant

Sometimes we change the program, but the final result does not change at all.



Example

Suppose a piece of code is never used. We change something inside that unused code — the program still gives exactly the same result. Our test cases cannot detect any difference.

EQUIVALENT MUTANT

A mutant that behaves the same as the original program.

These mutants can never be killed — no test case can tell them apart from the original.



Advantages



Helps us check the quality of test cases



Helps find missing test cases



Helps test boundary conditions



Gives better confidence in testing



Finds weak test cases



Improves software reliability



Useful for automated testing



Disadvantages



Time Consuming

A large program can create many mutants.



Expensive

Running thousands of tests requires more computing power.



Equivalent Mutants

They are difficult to identify.



Complex for Large Software

Large applications may have a huge number of possible mutations.



COMPARISON

Code Coverage vs Mutation Testing



Code Coverage

“How much of the code did my tests execute?”

Tests the code execution



Mutation Testing

“Can my tests find mistakes in the code?”

Tests the test quality

You may have 90% code coverage, but your tests may still be weak.



APPLICATIONS

Where Can We Use It?



Banking

ATM · Transactions · Payment systems



E-Commerce

Discounts · Payments · Shopping carts



Healthcare

Medical calculations · Patient systems



Automotive

Vehicle control systems



Mobile Apps

Login · Registration · Payment



Web Applications

Authentication · Business rules



WORKFLOW

Mutation Testing in Software Development

Mainly useful during Development and Testing:



Mutation testing sits right after “Run Tests” — it checks that those passing tests are actually trustworthy before you move on.



Simple Complete Example

ORIGINAL

```
int add(int a, int b) {  
    return a + b;  
}
```

MUTANT

```
int add(int a, int b) {  
    return a - b;  
}
```

Test: `add(5, 3)` — Expected: 8

RESULT



Original: **`add(5, 3) = 8`**



Mutant: **`add(5, 3) = 2`**

The test fails because the mutant's result doesn't match the expected value.

MUTANT IS KILLED



Why Is Mutation Testing Useful?

The main purpose is not to create bugs. It is to answer:

“Are my test cases good enough?”



Many mutants survive

→ Test cases are probably weak.



Most mutants are killed

→ Test cases are probably stronger.



Before I conclude

1

Mutation testing checks the quality of test cases.

2

A changed program is called a mutant.

3

If a test finds the change → Killed.

4

If a test does not find the change → Survived.

5

Mutation score tells us how effective our tests are.



CONCLUSION

Mutation Testing

An advanced software testing technique that creates small changes in a correct program and checks whether our test cases can find those changes.



If the test finds the change → the mutant is Killed.



If it does not find the change → the mutant Survives.

Mutation testing helps us identify weak test cases and improve software quality.

thank  you